Day 11 Homework: Two new tools for the help desk assistant

Mithran Sadasivam | AI Engineer Program, Phase 1 | Repository: github.com/mithransadasivam/FDE

What this homework was about

In class we built a help desk assistant that can use tools. The model reads a question, decides which tool it needs, and our code runs that tool and gives the result back. For homework I added two more tools, tested them, and then checked whether the model picks the right tool for different questions. The model I used is DeepSeek (deepseek/deepseek-chat) through OpenRouter. I only used the class database, which has made-up tickets and devices.

First, finishing the class code

Four parts of the class code were still empty, and the homework depends on them. So before the tasks I filled them in:

- **run_tool_loop**: asks the model, runs any tool it asks for, sends the result back, and stops after 6 steps.
- **validate_arguments**: checks the model's arguments (type, allowed values, limits, format) before a tool runs.
- **approve_and_run**: for tools that change data, asks the user first and writes every decision to a log.
- **The find_tickets schema**: the description the model reads to know when to use find_tickets.

The class tests for these all pass now.

Task 1: find_assets (a tool that only reads)

Goal: let the assistant answer "Which devices does this person have?" from the assets table.

What I did:

- Wrote **find_assets** in app/tools.py. You can search by username, department or device type, and you must give at least one.
- It opens the database **read-only**, so it cannot change anything.
- Values go into the query as **SQL parameters** (not pasted into the text), so a trick like *HR'; DROP TABLE assets* is just treated as a name.
- Wrote a schema with a clear description. The device type is limited to: laptop, desktop, monitor, docking station, phone. The username must look like *a.mensah*.
- Added it to the TOOLS list and to the assistant's instructions.
- Wrote **5 tests** with a temporary database: filters work, no filter gives an error, SQL tricks do nothing, bad arguments never reach the tool, and a fake model can choose the tool.

Result: I asked the real assistant: *Which devices does a.mensah have, and when was the laptop bought?* It made one find_assets call with assigned_to = a.mensah and answered:

```
tool call 1: find_assets({"assigned_to": "a.mensah"})
- Laptop IT-004512, Orbis L14 Gen 3, bought 2023-03-14
```

```
- Docking station IT-006078, Kestrel Dock 2, bought 2024-11-02
```

Task 2: assign_ticket (a tool that changes data)

Goal: let the assistant give an open ticket to a team, but only after a person approves, and log every decision.

What I did:

- Wrote **assign_ticket(ticket_id, team)**. The team must be one of: service desk, network team, end-user team.
- It refuses closed tickets ("INC-xxxx is closed; reopen it first") and tickets that do not exist.
- Marked it as **changes_data**, so the approval step from class protects it automatically.
- Added a plain sentence in describe_action(): *Give ticket INC-1013 to the end-user team*.
- Wrote **5 more tests** (10 in total in tests/test_my_tools.py): assigning an open ticket, refusing closed and unknown tickets, an unknown team never reaching the tool, a refusal changing nothing, and the approval sentence.

Refused run (nobody could approve, so it counts as no). The ticket stayed unchanged and this line went in logs/actions.jsonl:

```
APPROVAL NEEDED: Give ticket INC-1013 to the end-user team
(No one at a terminal to approve ...: treated as no.)

{"time": "2026-10-12T18:26:53-07:00", "tool": "assign_ticket",
 "arguments": {"ticket_id": "INC-1013", "team": "end-user team"}, "approved": false}
```

Approved run (answered yes). The ticket went from nobody to the end-user team:

```
{"done": true, "id": "INC-1013", "old_assignee": "nobody", "new_assignee": "end-user team"}

{"time": "2026-10-12T18:26:59-07:00", "tool": "assign_ticket",
 "arguments": {"ticket_id": "INC-1013", "team": "end-user team"}, "approved": true}
```

Note: I could not type "yes" in a real terminal while working in Claude Code. I ran the same approval step from a small script that answers yes. Afterwards I reset the database with *python -m scripts.make_helpdesk_db*.

Task 3: does the model choose the right tools?

Goal: write ten requests with the tools I expect, and measure how often the model picks them.

#	Request	Expected tool(s)
1	Hello! What can you help me with?	none
2	Is the Finance Portal down?	service_status
3	How many open Print Queue tickets are there?	find_tickets
4	What is the response target for a P1 incident?	search_policies
5	Is INC-1013 assigned to anyone? (sounds like a change, but is only a question)	get_ticket
6	What is 12 * 7.5 * 12?	calculate
7	Which devices does r.iyer have?	find_assets
8	Is the Finance Portal down, and how many open tickets are about it?	service_status + find_tickets
9	Which devices does a.mensah have and how many open SecureLink VPN tickets are there?	find_assets + find_tickets

10	Please give ticket INC-1014 to the network team	assign_ticket

Score before any change: 10 out of 10. Score after: 10 out of 10. The model chose the right tool every time, including for the question that only sounds like a change (it used get_ticket and did not try to change anything). I also tried five harder requests (for example the VPN time-out policy, "Who owns INC-1014?" and "Can you move INC-1015 to the service desk team?") and got 5 out of 5.

The description I changed: because nothing was WRONG, I had no failing request to fix. The one description I wrote was for **find_tickets**, which was only a placeholder ("TODO") in the class code. It now says to use it for questions about which tickets exist or how many there are, and to use get_ticket when one ticket is named. I also added lines about find_assets and assign_ticket to the assistant's instructions. The ten requests are saved in data/my_tool_tests.csv so I can re-run them whenever a description or the model changes.

Final check

- All unit tests pass: 210 passed (browser tests not included).
- The database is reset to the class data.
- Files: app/tools.py, app/tool_loop.py, tests/test_my_tools.py, data/my_tool_tests.csv, logs/actions.jsonl.